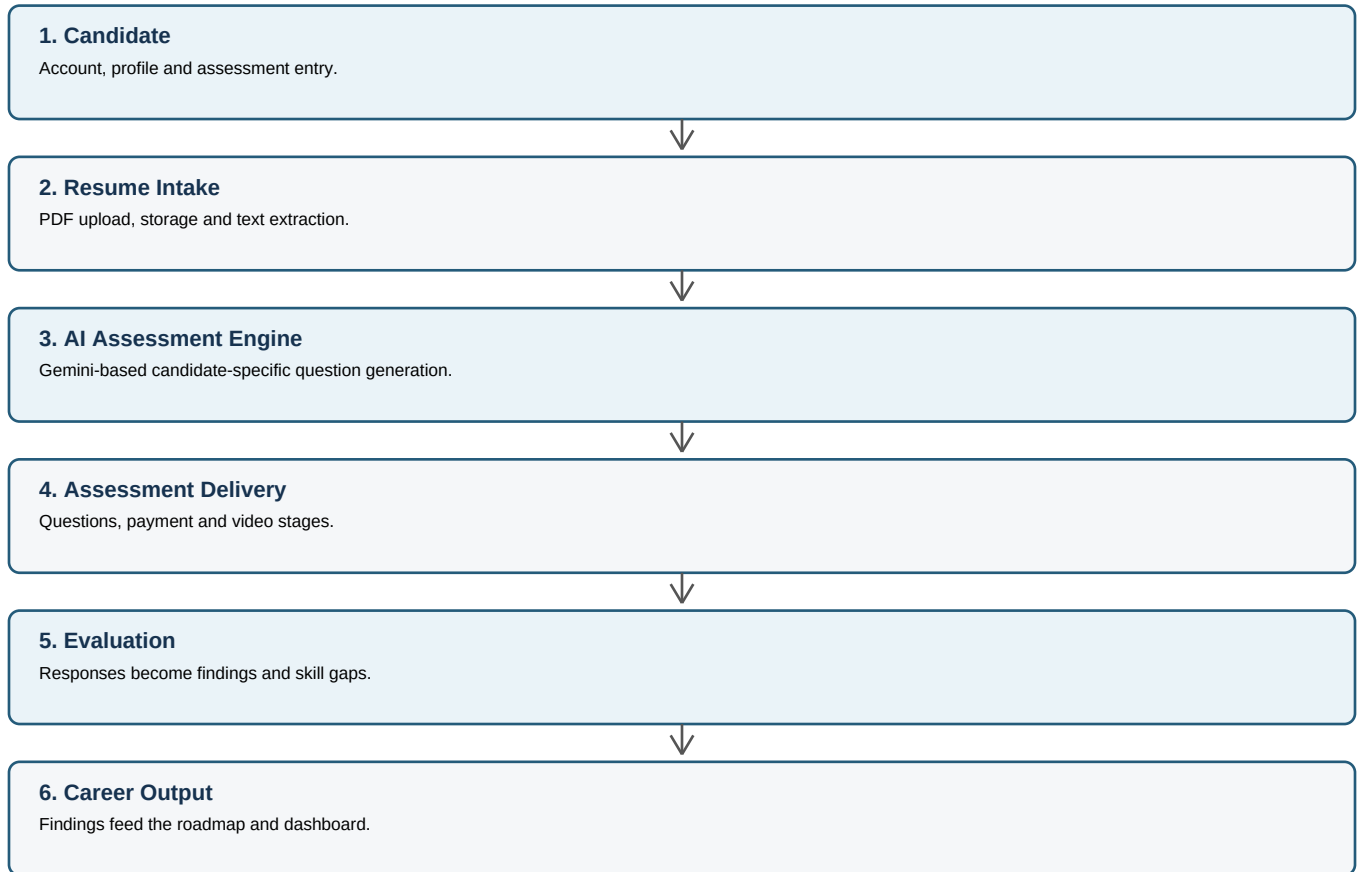


AI Career Assessment Platform

End-to-End Project Flow, Architecture, Tools & Technologies



Scope: based on the project structure and implementation details discussed in this project.

1. High-Level Architecture

Layer	Technology / Tool	Responsibility
Frontend	Next.js / React / TypeScript	Candidate UI, routing, forms, dashboards, assessment, findings and roadmap.
Backend API	Python + FastAPI	Authentication, resume upload, parsing, assessment orchestration, AI calls and business
Database	Relational DB + SQLAlchemy/Alembic	Users, resumes, assessments, questions, responses/results and schema migrations.
File Storage	Backend uploads / persistent storage	Uploaded resume files and metadata.
AI Layer	Google Gemini API / SDK	Candidate-specific question generation and AI analysis/evaluation.
Security	Backend security/configuration	Authentication, authorization and secret/configuration management.

External Services Payment + video/media capabilities Payment gating and video assessment stages where enabled.

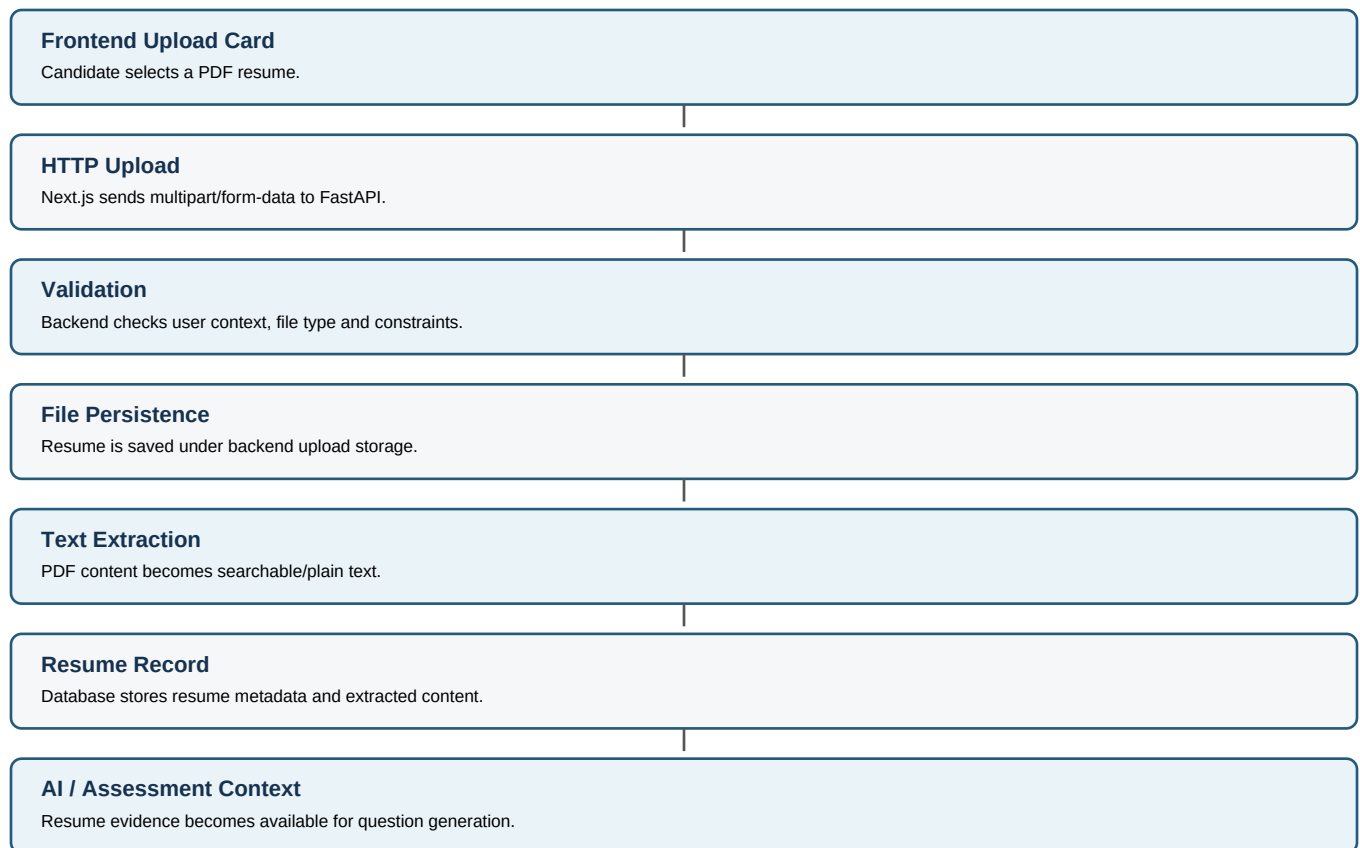
2. Technology Map

- Frontend: Next.js App Router, React, TypeScript and API integration.
- Backend: Python, FastAPI, schemas, services, configuration and security modules.
- Database: SQLAlchemy ORM with Alembic migrations over a relational database.
- AI: Google Gemini through the Python SDK.
- Resume processing: PDF upload and text extraction; API returns resume ID, filename, file type and extracted text length.
- Development: macOS, Python virtual environment and Git.

3. Candidate Journey — End-to-End Flow



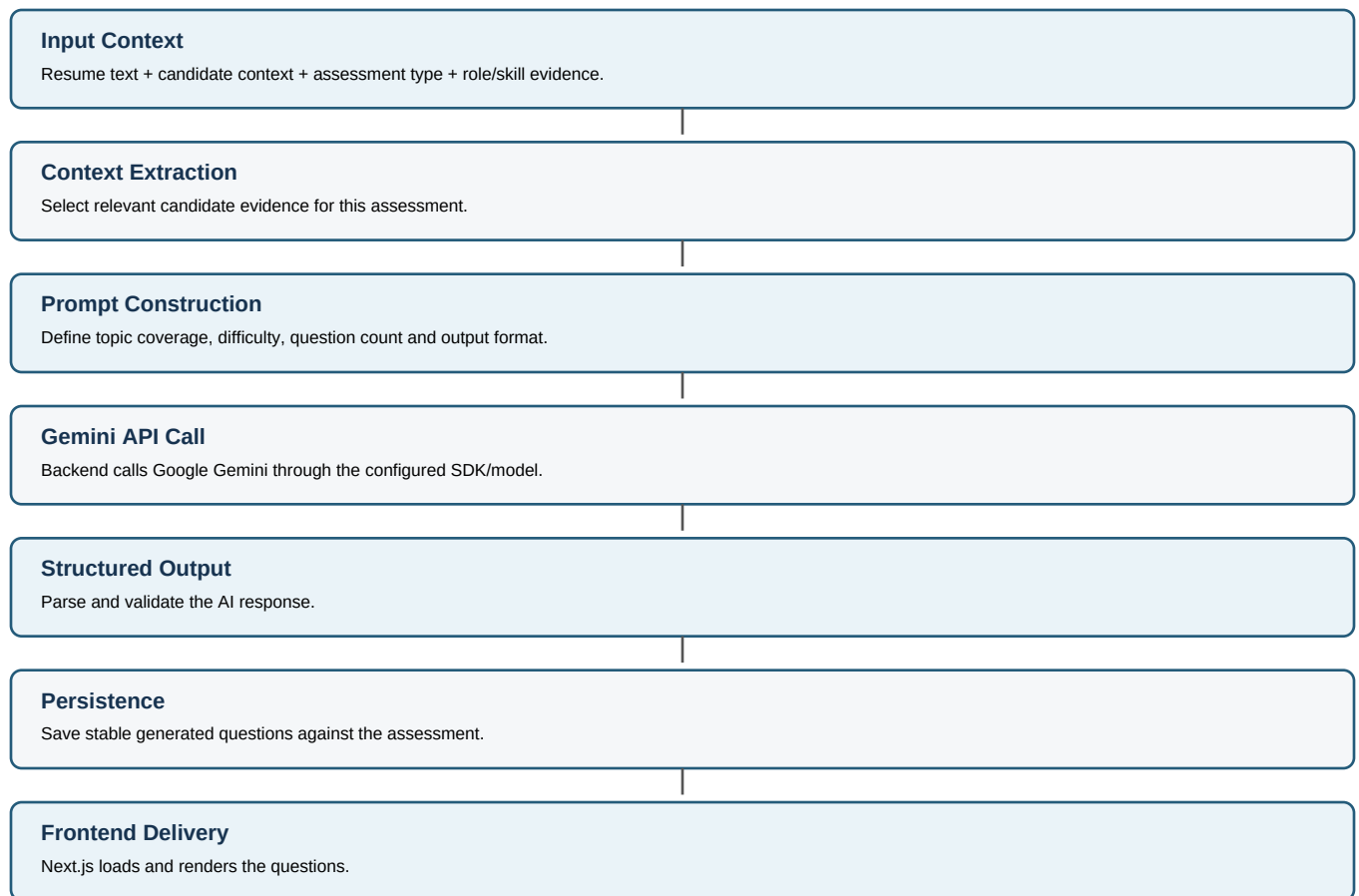
4. Resume Upload & Processing Flow



What happens after upload?

- The API response includes `resume_id`, `filename`, `file_type` and `text_length`.
- Extracted resume evidence can drive questions from skills, projects, work experience, education, certificates and target role.

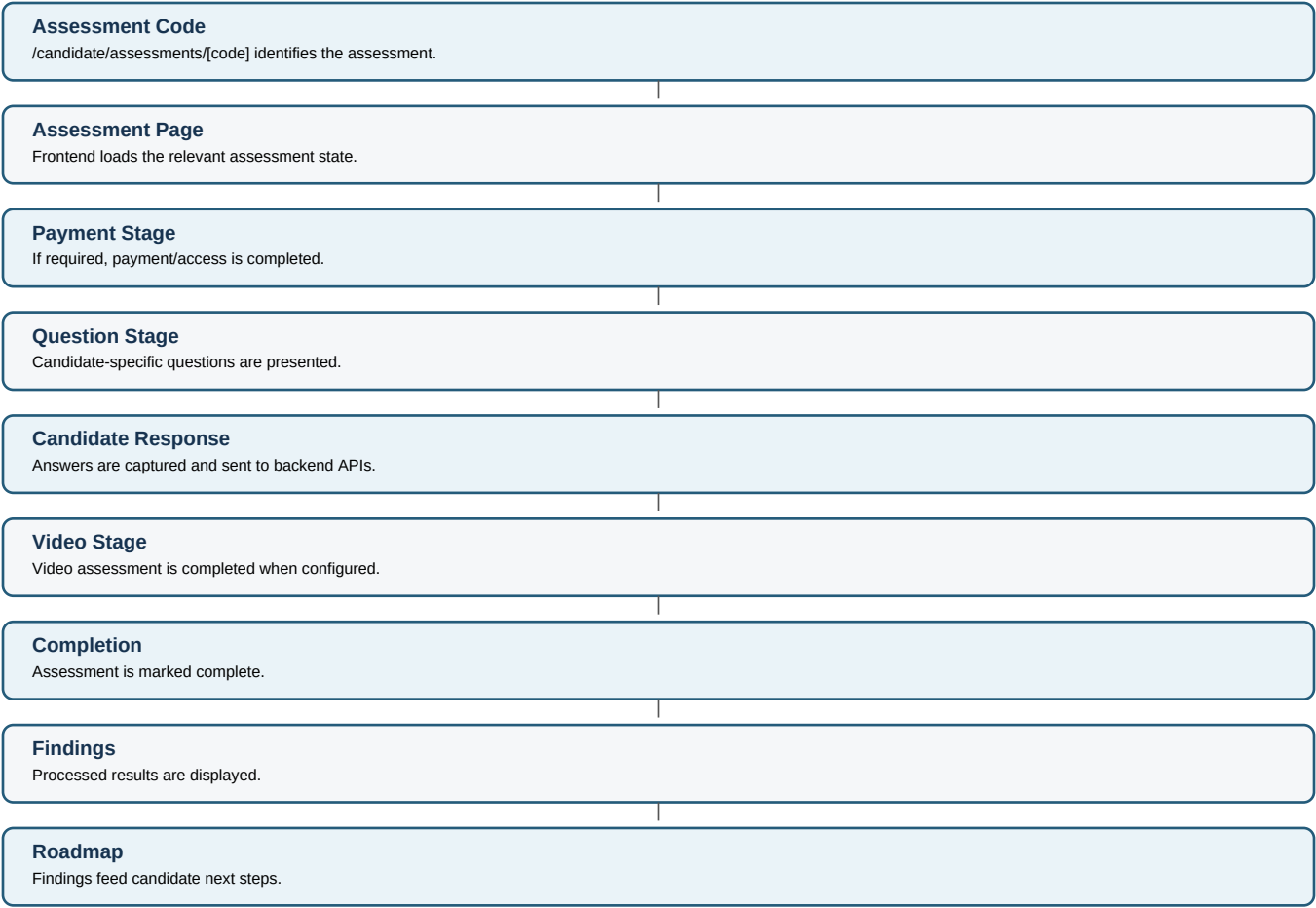
5. AI Question Generation Flow



Implementation Note

The current backend test showed Gemini's automatic-function-calling warning around direct `generate_content` usage. The warning recommends the chat interface for automatic function calling. This is an implementation detail and does not change the overall architecture.

6. Assessment Execution Flow



Frontend Route Map

/candidate/dashboard	Candidate dashboard and assessment status.
/candidate/profile	Candidate profile information.
/candidate/assessments	Assessment listing/entry point.
/candidate/assessments/[code]	Assessment-specific entry/state.
/candidate/assessments/[code]/payment	Payment/access stage.
/candidate/assessments/[code]/video	Video assessment stage.
/candidate/assessments/[code]/complete	Completion screen/state.
/candidate/assessments/[code]/findings	Assessment findings/results.
/candidate/roadmap	Career/learning roadmap.

7. Backend Internal Architecture

- app/main.py — FastAPI application entry point and router/application wiring.
- app/core/ — configuration and security.
- app/db/ — database connection/session and ORM models.
- app/schemas/ — request/response validation and API contracts.
- app/services/ — business logic including AI/assessment services.
- app/migrations/ — Alembic migration history, including resume schema changes.
- uploads/resumes/ — persisted uploaded resume files.

8. Database & Data Lifecycle



7. Backend Internal Architecture

API Layer — FastAPI

Receives frontend HTTP requests
Authentication / authorization • request validation • response serialization
Routes requests to service-layer business logic



Service Layer — app/services/

Resume parsing and text extraction • AI/Gemini integration • assessment generation
Scoring/evaluation • findings and roadmap generation • reusable business workflows



Data Layer — SQLAlchemy + Alembic

ORM models represent candidates, resumes, assessments, questions, responses and results
Alembic migrations keep database schema changes versioned and reproducible



Persistence & External Services

Relational database stores structured application data
Resume files live under uploads/resumes/<candidate_id>/
Google Gemini API/SDK supplies AI-generated assessment content and analysis

Request Flow

Frontend → FastAPI route → Service logic → DB / file storage / Gemini → Response → Frontend

Keep API routes thin; put reusable business logic in services; keep persistence concerns in the data layer.

11. Recommended AI Assessment Sequence

1. Load authenticated candidate and selected resume.
2. Read extracted resume text and assessment metadata.
3. Identify skills, projects, experience, education, certificates and role if present.
4. Select balanced topics so one repeated keyword does not dominate.
5. Build a strict Gemini prompt with difficulty, count and output schema.
6. Call Gemini server-side; keep credentials out of the frontend.
7. Parse and validate the response.
8. Reject/regenerate malformed output.
9. Persist generated questions for stable delivery.
10. Store responses and pass them to findings/evaluation.

12. Current Project Status

- Candidate frontend routes exist for dashboard, profile, assessments, payment, video, completion, findings and roadmap.
- Backend has a layered FastAPI structure with core, database, schemas, services and migrations.
- Resume upload/extraction is working and returns resume metadata plus text length.
- Uploaded resume files are stored under the backend resume upload directory.
- A target-role field has been introduced in the resume/database layer.
- Gemini integration is present in the backend service/test path.
- Main remaining orchestration: resume evidence → AI questions → assessment persistence → response evaluation → findings → roadmap.

13. One-Page Mental Model

Candidate → Next.js → FastAPI → Resume Parser/Storage → Database
→ AI Context Builder → Gemini → Questions → Assessment UI
→ Responses → Evaluation → Findings → Roadmap → Candidate

Gemini is one component, not the whole application. FastAPI orchestrates the workflow, the database maintains state, resume processing supplies candidate evidence, and Next.js provides the user interaction layer.

14. Suggested Completion Order

1. Stabilize resume upload + extraction.
2. Finalize resume-to-context extraction.
3. Stabilize Gemini question generation and structured output.
4. Persist generated questions to the assessment.
5. Connect assessment UI to question/answer APIs.
6. Complete response persistence and evaluation.
7. Connect findings page to real backend results.
8. Connect roadmap generation/display to findings.
9. Test login → resume → assessment → completion → findings → roadmap.
10. Add final authorization, error handling and deployment configuration.